Avdhut Ramish Madhavi
SE-C2 C60 TU3521271005

Terna Engineering College, Nerul

8/10

Assigament No. 2

Q1. → N-Queen Problem and Sum of Subsets

A) N-Queen problem using backtracking

The N-Queen problem is to place 8 queen on an 8x8 chessboard so that no two queen attack each other. NO two queen can be in the same row, colown or diagonal.

Backtracking approach:

1. Place one queen in the first row.
2. Check whether the chosen position is safe
3. If safe, move to the next row and place another column.
4. In no safe position is available, remove the previous queen and try another
5. Repeat until all 8 queen are placed.

The column positions for rows 1 through 8 are:

[1, 5, 8, 6, 3, 7, 2, 4]

Here, the number in each position represents the column containing the queen in that row

Terna Engineering College, Nerul

	1	2	3	4	5	6	7	8	9
1	Q1								
2					Q2				
3								Q3	
4						Q4			
5				Q5					
6							Q6		
7		Q7							
8				Q8					

B) Sum of subset using Backtracking.

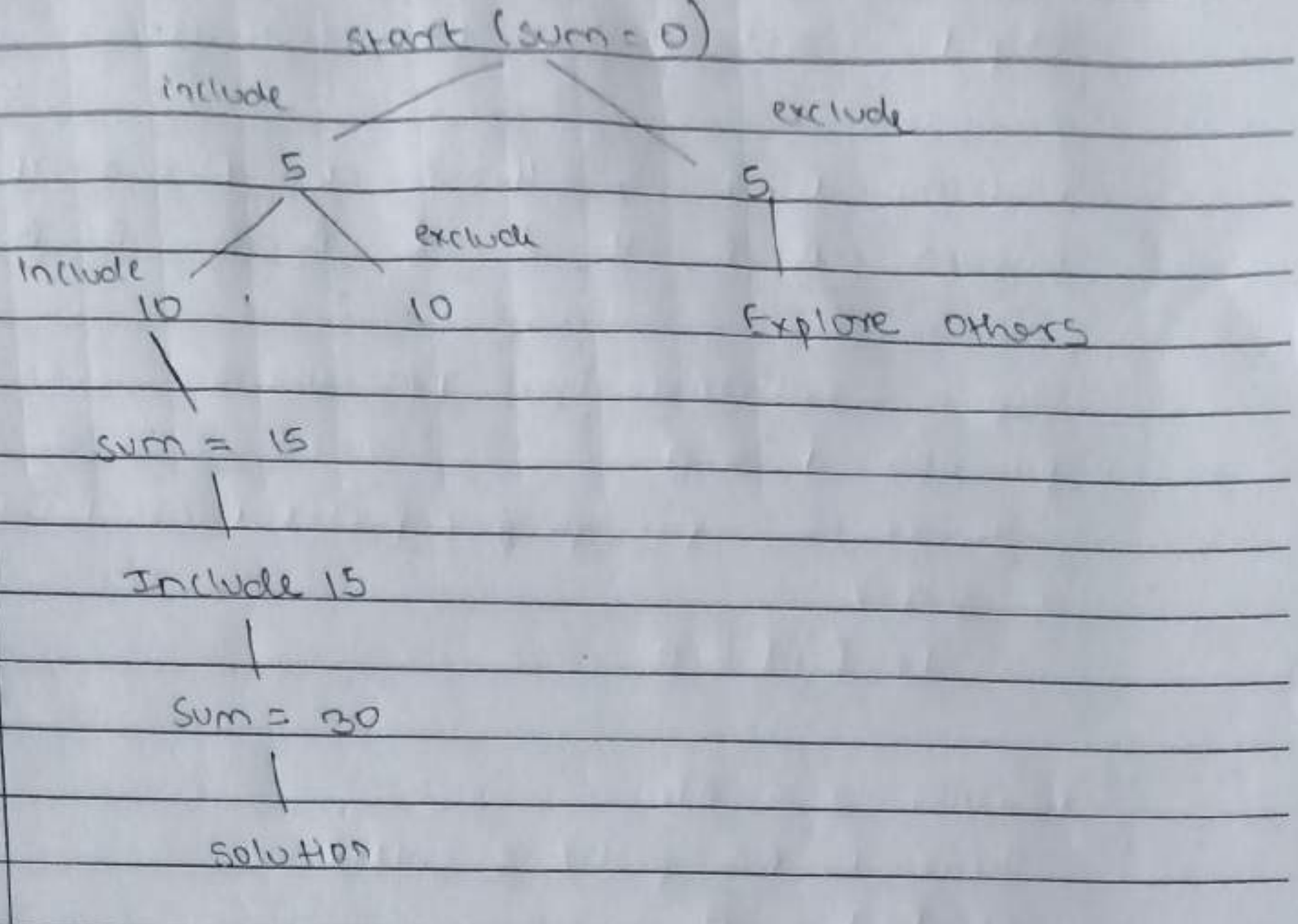
Given :-

$S = \{5, 10, 12, 13, 15, 18\}$, $M = 30$

Method :- For each element, decide whether to include or exclude it. If the sum exceed 30 stop exploring that branch. If the sum equal 30, record the subset.

The subset are :-

1. $\{5, 10, 15\}$ because $5+10+15 = 30$
2. $\{5, 12, 13\}$ because $5+12+13 = 30$
3. $\{12, 18\}$ because $12+18 = 30$
4. No other subset of the given set has a sum equal to 30



Another succesfull path is to include 5, 12, 13 A third solution is obtained by including 12 & 18.

Q2. Knuth - morris prath (kmp) & Robin karp string matching.

→

Given :-

Text : ABABDABACDABABCABAB

pattern :- ABABCABAB

i) Knuth - Morris Pratt (KMP)

KMP is a string matching algorithm that uses an LPS array to avoid rechecking characters that have already matched.

Step 1: Construct the LPS array

Position	1	2	3	4	5	6	7	8	9
Pattern	A	B	A	B	C	A	B	A	B

Step 2: calculate the LPS value

Position 1 = A

There is no proper prefix that is also a suffix

LPS [1] = 0

Position 2 :- AB

There is no prefix & suffix

LPS [2] = 0

Position 3 : ABA

There is prefix & suffix are longly match. with A.

LPS [3] = 1

Position 4 :- ABAB

There is longest matching prefix & suffix with AB

LPS[4] = 2

Position 5 : ABABC

Non empty prefix match.

LPS[5] = 0

position 6 :- ABABCA

The longest matching prefix & suffix.

LPS[6] = 1

position 7 : ABABCAB
The longest matching prefix & suffix
LPS [7] = 2

position 8 :- ABABCABA
match found
LPS[8] = 3

Position 9 :- ABABCABAB
The longest match found
LPS[9] = 4

Step 3 :- write the final LPS array.

pattern	A	B	A	B	C	A	B	A	B
LPS val	0	0	1	2	0	1	2	3	4

ii) Rabin Karp algorithm.
Robin Karp algorithm searches for a pattern by calculating a hash value for the pattern and for text window of same lengths.

Working :-
1. Calculate the hash value of the pattern.
ABABCABAB
2. Calculate the hash of the first nine characters of text.

3 Slide the window 1 characters at a time and update the hash using a rolling hash formula.
4. If a window hash equals the pattern hash; compare the actual characters to confirm the match.
5 Report the match at position 11.

Difference between KMP & Robin Karp.

KMP	Robin Karp.
i. Uses an LPS array	Uses rolling hash value
ii Avoid unnecessary comparisons through prefix information.	use hash comparisons to identify candidate window.
iii. worst case time is $O(n+m)$	worst case time is $O(m \cdot n)$

Q3 Complexity classes: P, NP, NP-Hard, and NP-complete

→ A. Define.

i) P(Polynomial time)

P is the class of decision problem that can be solved in polynomial time by a deterministic algorithm.

ii) NP(Non deterministic Polynomial time)

NP is the class of problems whose given solution can be verified in polynomial time by a deterministic algorithm.

iii) NP Hard

A problem is NP-Hard if every problem is NP can be reduced to it in polynomial time. A NP Hard problem does not necessarily belong to NP.

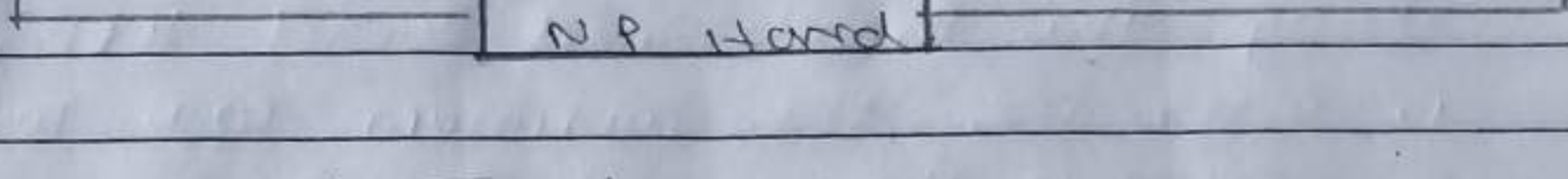
iv) NP-complete

A NP complete problem is a problem that belong to the class NP and is also NP-Hard There are considered the hardest problem in the class NP.

BY classification of the given problem

Problem	complexity class	Justification
1. Finding the shortest path between two vertices in a grap.	P	can be solved in polynomial time using algorithms such as unweighted graph or Dijkstra algorithm for non +ve edge weights
2. Travelling salesman problem	NP Hard & NP complete.	Finding the min cost tour is NP Hard deciding wether a tours of cost at most k exists in NP complete.
3. 0/1 knapsack problem	NP Hard & NP complete	The optimization version is NP Hard the decision version is NP complete.
4. checking wether a boolean formula has a satisfying assignment	NP - complete.	SAT belong to NP and is NP - Hard.

⇨ Diagram showing the relationship between complexity class.



In ~~standard~~ Notation

$$P \subseteq NP$$

$$NP\ complete = NP \cap NP\text{-}Hard$$

It is not known whether $P = NP$. If any NP complete problem can be solved in polynomial time then $P = NP$

Q4. Travelling salesperson Problem.

A salesperson has to visit four cities A, B, C, D exactly once and return to the starting city.

Consider the following cost matrix.

From / to	A	B	C	D
A	-	10	15	20
B	10	-	35	25
C	15	35	-	30
D	20	25	30	-

Using the Branch and Bound technique.

1. Determine the minimum cost tour
2. Calculate the total cost of the selected tour.
3. Briefly explain how Branch-Bound helps eliminate unnecessary paths.

objective : find the tour having the minimum total cost using the Branch and bound technique.

Step 1 :- Fix the starting city.

without loss of generality ; fix city A as the starting city. The salesperson must visit B, C, D and then return to A

madhavi C2 Batch

U352627005

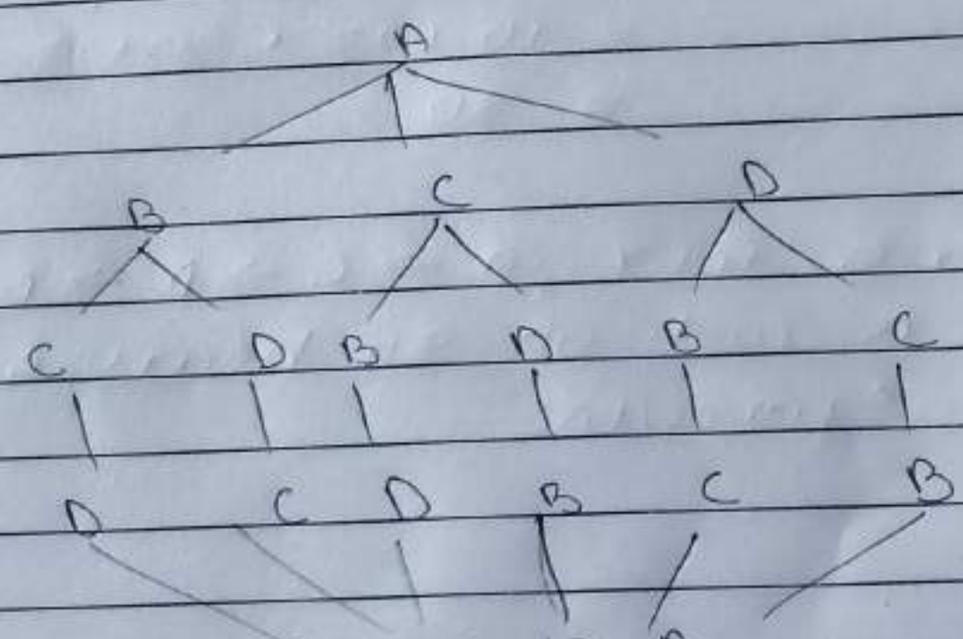
Engineering College,Nerul

Terna Engineering College,Nerul

Step 2: Construct the possible tours since the starting city is fixed. there are 3! = 6 possibble tours.

NO.	TOUR	cost cal	Total cost
1	A → B → C → D → A	10+30+35+20	95
2	A → B → D → C → A	10+25+30+15	80
3	A → C → B → D → A	13+35+25+20	95
4	A → C → D → B → A	15+30+25+10	80
5	A → D → B → C → A	20+30+25+15	95
6	A → D → C → B → A	20+30+35+10	95

STEP 3: Apply Branch and Bound the possible tours can be represented as a search tree



- The best complete tour found has cost 80
- Any practical path whose bound is at least 80 can be pruned, because it cannot produce a strictly better solution
- Continue exploring any branch whose lower bound less than 80 until the minimum is established.

Step 4:- calculate the minimum cost tour

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

The total cost is :-

Total cost = c(A,B) + c(B,D) + c(D,C) + c(C,A)

= 10 + 25 + 30 + 15

= 80

The reverse tour $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$ also has a cost of 80 because the cost matrix is symmetric

Step 5:- How Branch & Bound eliminates unnecessary paths.

1. Branching :- Divide the problem into smaller such problems by selecting the next city.

2. Bounding :- Calculate a lower bound on the minimum possible cost of completing each partial tour.

3. Pruning :- If a branch's lower bound is greater than or equal to best known complete-tour cost discard it because it cannot produce a strictly better answer.

4. Selection :- Explore promissing branches until the minimum cost tour is established.